Advanced CPU Architecture & Hardware Accelerators

LAB5 preparation report

Scalar Pipelined RISC-V CPU µArchitecture

Hanan Ribo

08.06.26

Table of contents

1. Aim of the Assignment

- Design, synthesis and analysis of a simple RISC-V compatible CPU.
- Scalar Pipelined RISC-V CPU µArchitecture (from single-cycle RISC-V CPU µArchitecture)
- Understanding of CPU vs. MCU concept.
- Understanding FPGA memory structure.

2. Definition and prior knowledge

The aim of this laboratory is to design a Scalar Pipelined RISC-V CPU from a single-cycle RISC-V compatible CPU. The CPU core must be capable of performing instructions from a given RISC-V instruction set. The design will be executed on the Intel-FPGA Board. The RISC-V architecture is Harvard architecture to increase throughput and simplify the logic. **You are required to support the Assembly Instruction Set of RV32IM**. For additional information regarding RISC-V CPU, Architecture, and ISA, see RISC-V technical documents.

3. Assignment definition

The architecture must include a RISC-V ISA compatible with ITCM and DTCM memories for hosting code and data, respectively. The block diagram of the architecture is given in Figure 1. The CPU will have a standard RISC-V register file. The top level and the RISC-V core must be structural. The design must be compiled and loaded onto the Intel-FPGA board for validation. A single clock (***clk_i***) should be used in the design.

Note: You must use push-button KEY0 as a core RESET (to bring the PC to the first program instruction).

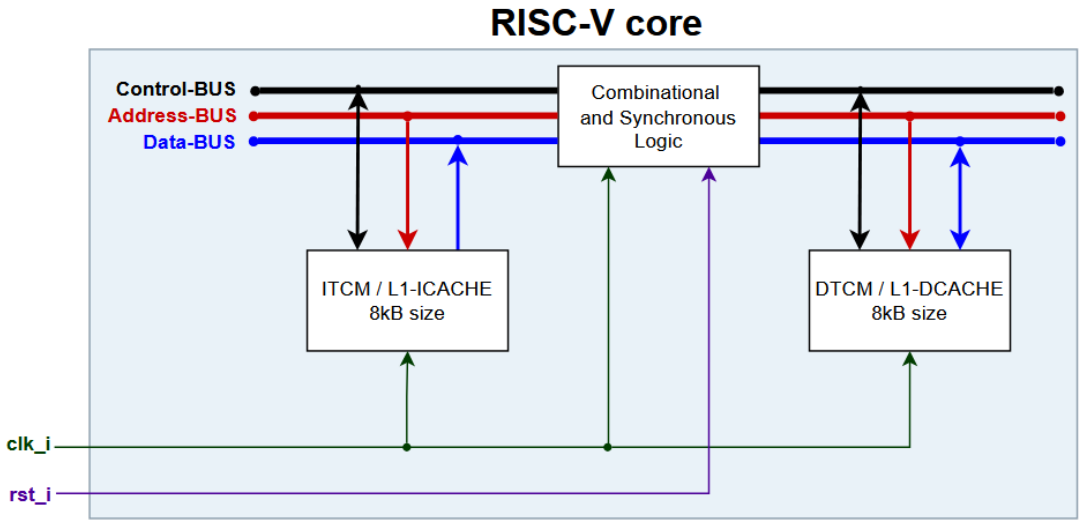

Figure 1 : System architecture

4. Task part1 - Scalar single-cycle RV32IM µArchitecture

You are given a single-cycle RV32I µArchitecture, as shown in *Figure 2,* and its subpart division in *Figures 3* and *4*. The LAB5 task part 1 is to elevate the given single-cycle RV32I µArchitecture to a single-cycle RV32IM (MULDIV partial) µArchitecture. M-extension stands for MULDIV operations, but the task requirement is to support only the ***mul*** instruction (from the multiplication set ***mul, mulh, mulhsu, mulhu***), as shown in Figure 5. The instruction ***mul*** *rd, rs1, rs2* multiplies the lower 16-bit (half-word) of registers rs1, rs2 and the result of 32-bit is written to register rd. The required algorithm for 16-bit multiplication is based on using FPGA's four embedded 8-bit multipliers as shown in Figure 6.

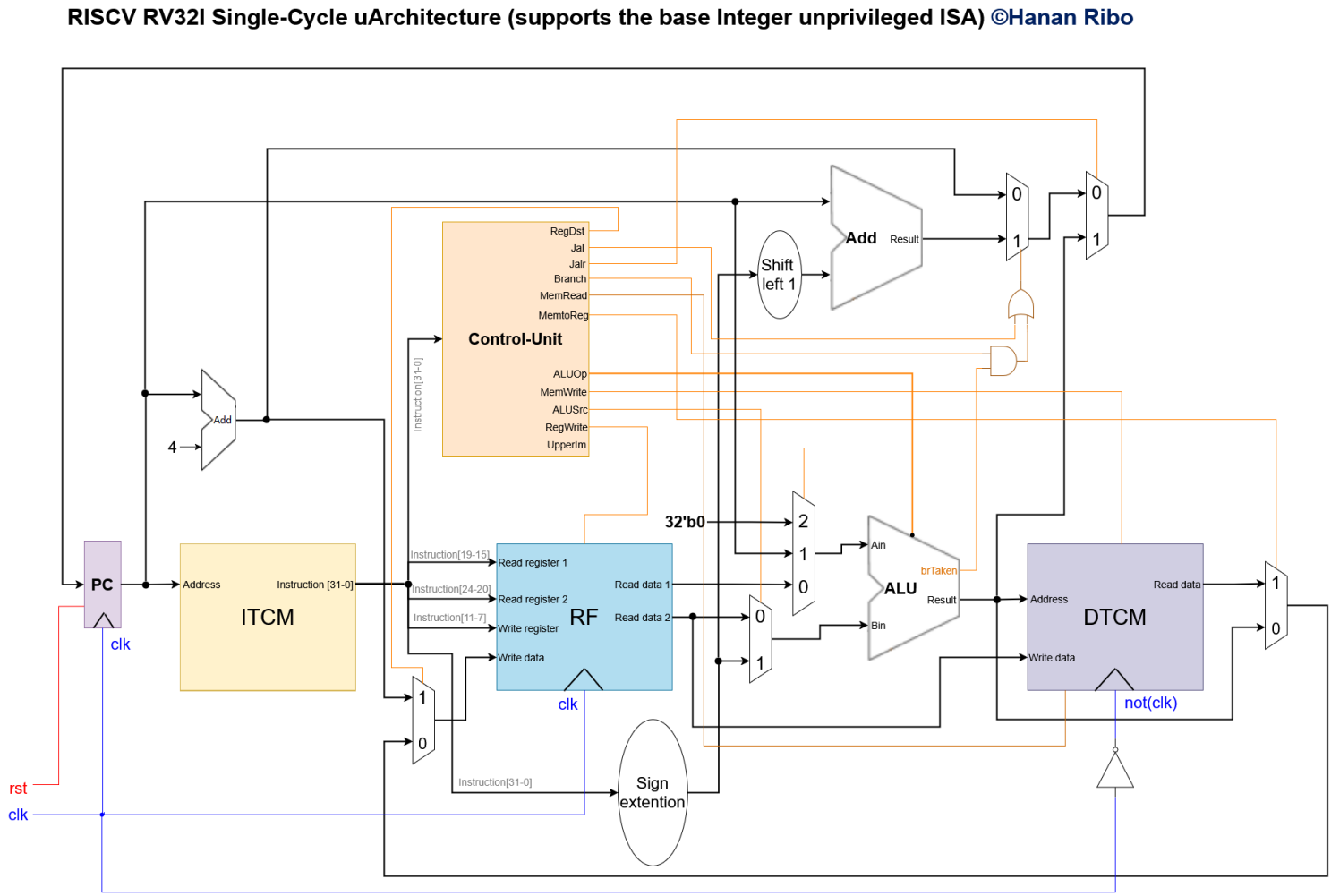

Figure 2: Single-cycle RV32I µArchitecture

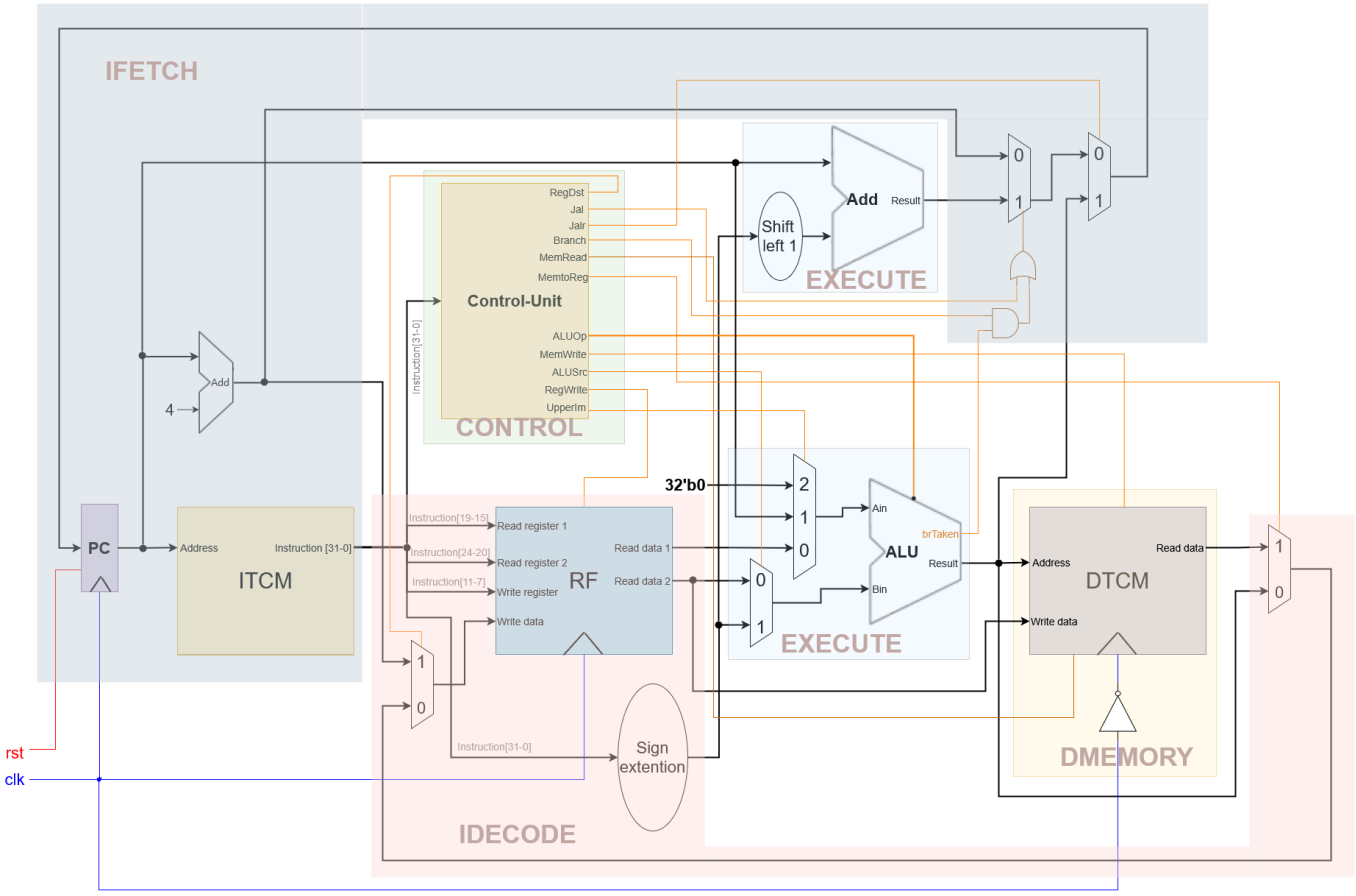

Figure 3: Single-cycle RV32I µArchitecture's submodules

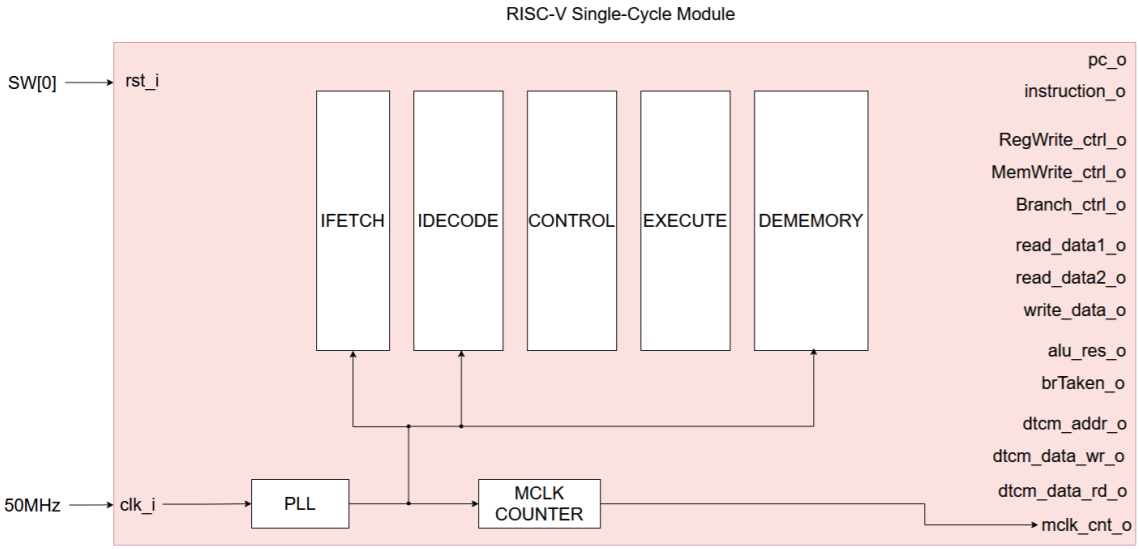

Figure 4: Single-cycle RV32I µArchitecture top entity

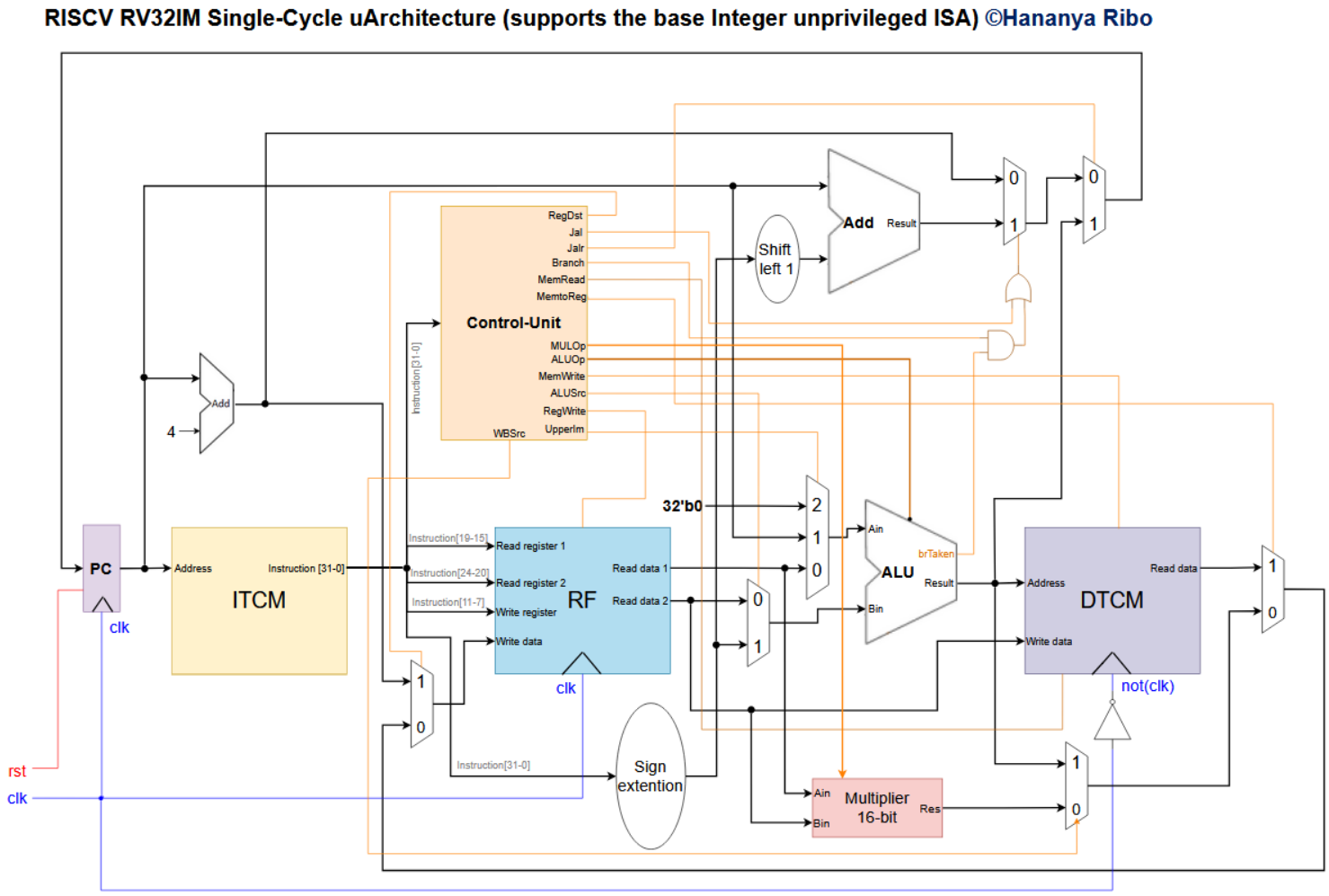

Figure 5: Single-cycle RV32IM (MULDIV partial) µArchitecture

Logical equation of 16-bit multiplication for RESULT = A × B:

Stage 1

- P0 = A_low × B_low
- P1 = A_low × B_high
- P2 = A_high × B_low
- P3 = A_high × B_high

Stage 2

- M = P1 + P2
- RESULT = P0 + (M << 8) + (P3 << 16)

Figure 6: 16-bit multiplication using four 8-bit multipliers

5. Task part2 - Scalar pipelined RV32IM µArchitecture

The Single-cycle RV32IM (MULDIV partial) µArchitecture you have designed in task part 1 needs to be elevated to a pipelined RV32IM (MULDIV partial) µArchitecture, shown in Figure 7. The development process is going through intermediate phases of slicing the single-cycle µArchitecture into 5 stages. Figure 7 shows a pipelined RISC-V architecture with Stall detection Logic based on a Combinational Check approach (versus score-board approach) as a Data Hazard detection unit. Data Forwarding Unit is supported. Conditional branches and unconditional jumps are resolved in the pipeline's stage 4.

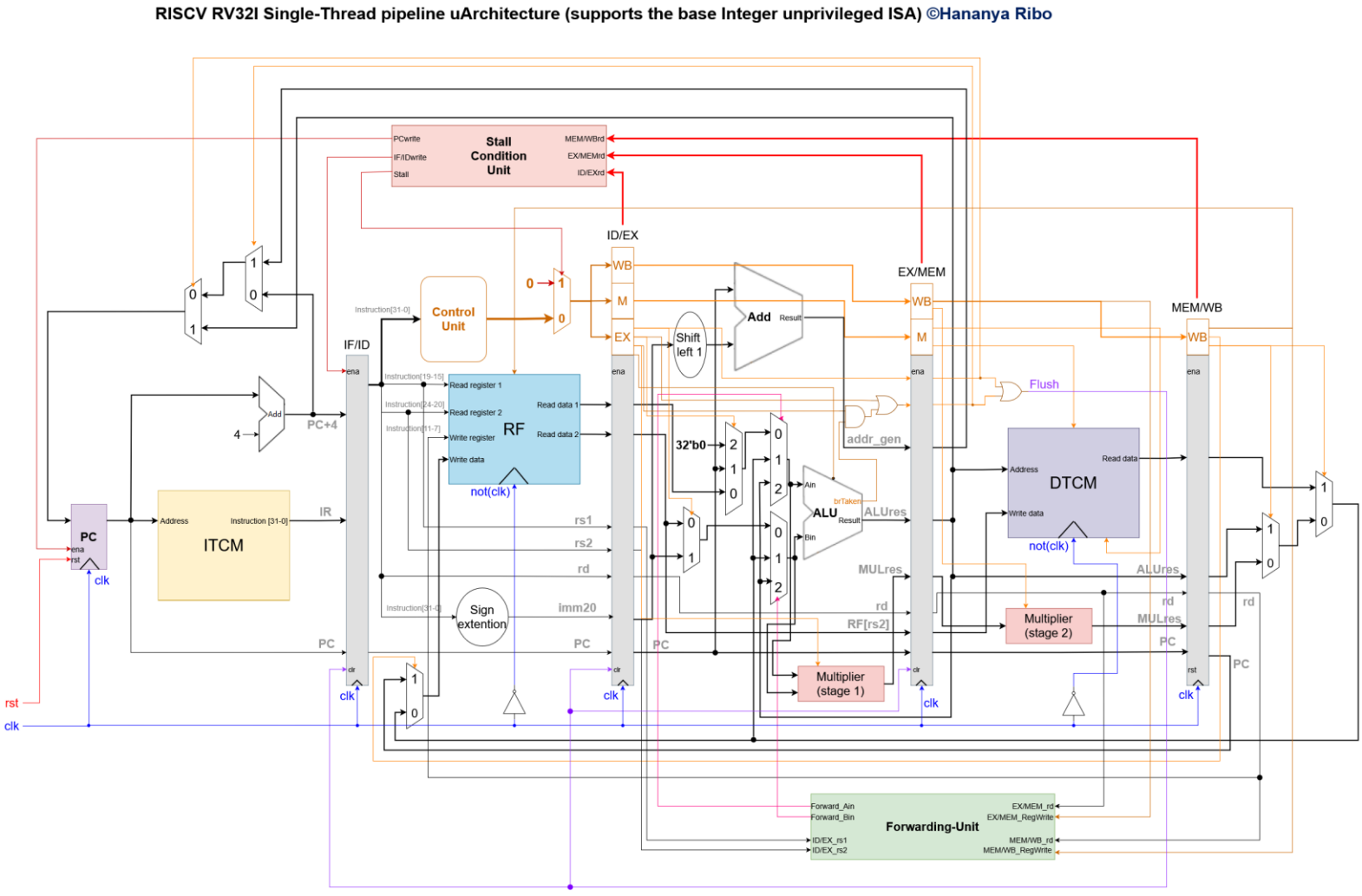

Figure 7: five-stage pipelined RV32IM (MULDIV partial) µArchitecture with interlocking check and full forwarding support

6. System design flow

For design validation using signal tap chosen signals must be added to the top entity as shown in Figure 4 for single cycle µArchitecture and for pipeline µArchitecture as shown in Figure 8. The following added signals if Figure 8 are essential for supporting applications' IPC calculation and breakpoint debug ability.

i. **STCNT_o** (stall counter register) 8-bit register – **STCNT_o** cleared on reset:

This register is a core stalls counter that counts on the clock rising edge when a stall occurs.

ii. **FHCNT_o** (flush counter register) 8-bit register – **FHCNT_o** cleared on reset:

This register is a core flush counter that counts on the clock rising edge when a flush occurs.

iii. **BPADDR_i** (break point address) 8-bit register:

This register uses a breakpoint address (of word granularity) to feed logic that issues a signal-tap trigger event. This signal-tap trigger enables tracking CPU core signals from any PC address until the signal tap buffer is full. **BPADDR_i** is cleared on reset. *Using the Auto-run button in the signal tap window, we can debug the CPU core as much as we wish in only one signal tap session*. **BPADDR_i** register is fed by SW7-SW0 located on the FPGA board. The Signal tap trigger equation is:

a) $\boldsymbol{SignalTaptrigger = (IF_{PC} == BPADDR\_i)}$

b) $\boldsymbol{IPC = \frac{\text{CLKCNT\_o} - (STCNT\_o + 4 + \text{depth} * \text{FHCNT\_o})}{\text{CLKCNT\_o}} = \frac{InstructionCounter}{\text{CLKCNT\_o}}}$ **where: IPC=1/CPI**

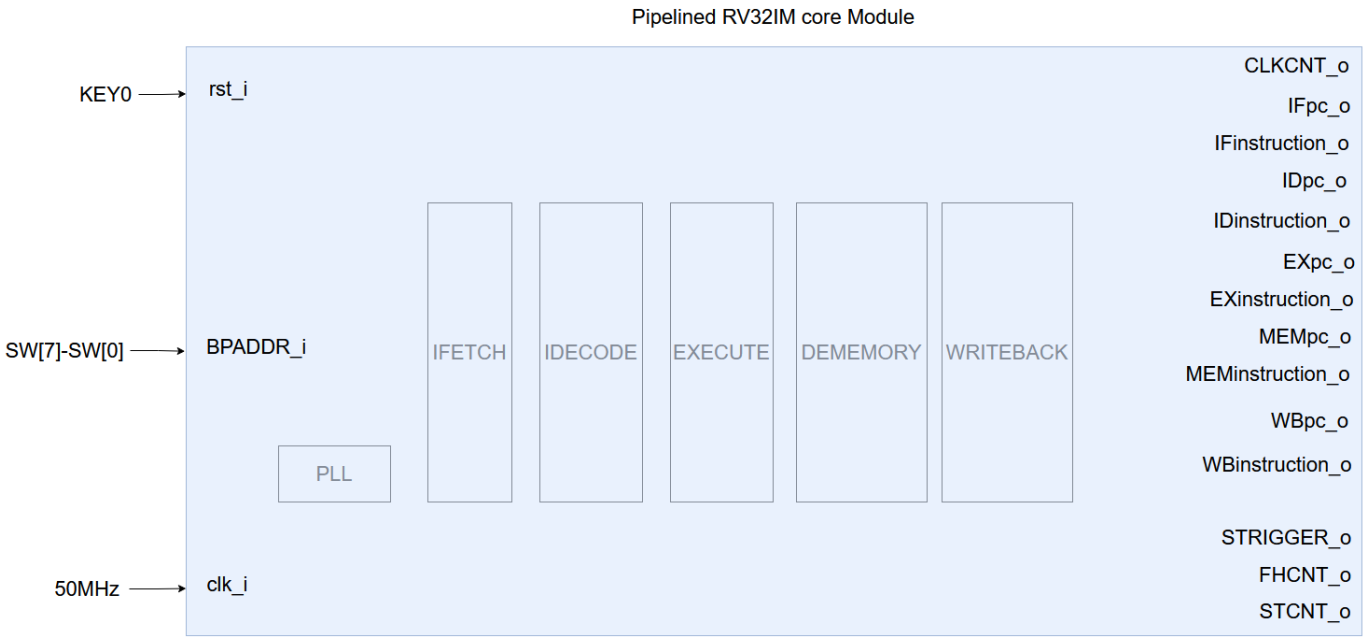

Figure 8: Pipelined RISC-V architecture top entity

Note: The capability of Signal-Tap validation in terms of channel number versus channel depth is given under the total memory condition equation:

$$STdept = \frac{Embedded\ memory\ size - Design\ usage\ memory\ zise}{\#STchannels}$$

7. PPA characterization of RV32IM (MULDIV partial) designs:

Area						
	Logic elements	Registers	I/O pins	Embedded memory bits	Embedded 9-bit Multipliers	PLLs
Single-Cycle RV32IM						
Pipelined RV32IM						

Note: Attaching the print screen of the Quartus Arae report is mandatory

Performance			
	f_{max}	Critical path (what is the slowest submodule and why does it cause the critical path)	f_{MCLK} $(= f_{sysclk})$
Single-Cycle RV32IM			
Pipelined RV32IM			

Note: Attaching the print screen of the Quartus *fmax* report is mandatory

Power				
	Total power consumption	**Static** power consumption	**Dynamic** power consumption	**I/O** power consumption
Single-Cycle RV32IM				
Pipelined RV32IM				

Note: Attaching the print screen of the Quartus Power report is mandatory

Note:
A qualitative design is discernible by the three PPA values. Highest *fmax* closest to the theoretical value $fmax_{pipeline} = 5 * fmax_{single\ cycle}$. The lowest area (number of LEs and Registers in an FPGA) depends on optimal design considerations, ditto for the lowest power consumption.

8. RV32IM core design flow based on benchmark applications

a) Characterize the architecture design in detail

b) Write HDL code of the architecture design

c) Verify your design using ModelSim based on benchmark applications.

i. As a golden model, at the end of the application run, compare the RARS output file DTCM.h with the ModelSim output file DTCM.mem.

ii. For full coverage, use basic and advanced benchmark applications.

iii. Check the IPC by comparing the two sides of the equation in clause 6.iii.b (IPC accuracy matters).

iv. In case of inequality in ii or iii, **return to phase (b).**

d) Use Quartus with *.sdc file for PPA analysis (see three tables in clause 7).

e) Validate the design using ISMCE and Signal-Tap validation tools.

i. As a golden model, at the end of the application run, compare the RARS output file DTCM.hex with the ISMCE output file DTCM.hex.

ii. For full coverage, use basic and advanced benchmark applications.

iii. Check the IPC by comparing the two sides of the equation in clause 6.iii.b (IPC accuracy matters).

iv. In case of inequality in ii or iii, **return to phase (b).**

9. LAB5 report file requirements

The following must be presented in the **Report_lab5.pdf** report file.

a. Top-level block review diagram of your design.

b. RTL Viewer results

c. Three tables in clause 7 with their shot screen report (from Quartus).

d. Short description of each HDL source file.

e. Documentation Style - Content with page numbers, images, and tables will be numbered. The caption of an image and tables below the image or table.

f. Elaborated analysis and wave forms:

A basic waveform for benchmark applications test4-test1to explain the system timing.

g. Submission requirements:

- A ZIP file in the form of **id1_id2.zip** (where id1 and id2 are the identification number of the submitters, and id1 < id2) *must be uploaded to Moodle only by the student with id1* (any of these rule violations disqualifies the task submission).

- The **ZIP** file will contain the next six subdirectories (***only the exact next subfolders***):

Directory	Contains	Comments
DUT	Two subfolders, each containing design VHDL files: • **RV32IM_sc** • **RV32IM_pipeline**	Only the design VHDL files (excluding test bench). Note: your project files must be well compiled (in ModelSim and Quartus separately) without errors as a basic condition before submission
TB	Two subfolders, each containing the design testbench file: • **RV32IM_sc** • **RV32IM_pipeline**	• In folder **RV32IM_sc,** insert the **tb_RV32IM_sc.vhd** file for RV32IM single-cycle design. • In folder **RV32IM_pipeline,** insert the **tb_RV32IM_ pipeline.vhd** file for RV32IM pipeline design.
SIM	Two subfolders, each containing the ModelSim *.do file: • **RV32IM_sc** • **RV32IM_pipeline**	
DOC	Project documentation	• **Readme.txt** description of the content of each folder (and subfolder) for user convenience navigation. • **Report_lab5.pdf** full report file described in clause 9
Quartus	Two subfolders, each containing a Signal-Tap file, an SDC file, and a SOF file: • **RV32IM_sc** • **RV32IM_pipeline**	Do not place files that are not relevant for compilation or a result of compilation!

Table 1 : Directory Structure

10. Grading policy

Weight	Task	Description
10%	Documentation	The "clear" way in which you presented the requirements, the analysis, and the conclusions on the work you've done
90%	Analysis and Test	The correct analysis of the system (under the requirements)

Table 4: Grading